Package 'gwkit'

July 11, 2026

Type Package

Title Convenience Wrappers and Workflows for Geographically Weighted Analysis

Version 0.0.0.9000

Author Francis Tsiboe [aut, cre] (<https://orcid.org/0000-0001-5984-1072>), TERRA ANALYTICS LLC [cph, fnd]

Maintainer Francis Tsiboe <ftsiboe@hotmail.com>

Contributor -

Reviewer -

Creator Francis Tsiboe

Copyright TERRA ANALYTICS LLC (Kansas, USA)

Description A toolkit of convenience wrappers and reproducible workflows around 'GWmodel' and related spatial packages ('sf', 'terra', 'sp') for geographically weighted analysis. Provides point-, polygon-, and county-level geographically weighted summary-statistics pipelines, distance-metric presets, and utilities to reduce results estimated across a grid of kernel and distance-metric settings to a single per-unit consensus, with optional Queen-contiguity reconciliation. The package wraps, and does not replace, the underlying estimators.

License GPL-3

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.1.0)

Imports data.table,
GWmodel,
sp,
sf,
terra,
dplyr,
urbnmapr,
methods,
stats,
utils

Suggests testthat (>= 3.0.0),
knitr,
rmarkdown

Remotes github::UrbanInstitute/urbnmapr

URL https://github.com/ftsiboe/gwkit

BugReports https://github.com/ftsiboe/gwkit/issues

Config/testthat/edition 3

Contents

`estimate_gwlag_by_point` *Geographically weighted spatial lag (neighbour-weighted mean) at points*

Description

For each target unit, computes the geographically weighted mean of one or more value columns over the source units, using `GWmodel::gw.weight()` weights on a `GWmodel::gw.dist()` distance matrix. With `include_self = FALSE` the focal unit is excluded from its own neighbourhood, yielding a strict neighbour mean (a spatial lag).

Usage

```
estimate_gwlag_by_point(
  data,
  unit,
  value_cols,
  coords = c("longitude", "latitude"),
  predict_data = NULL,
  distance_metric = "Euclidean",
  kernel = "bisquare",
```

```
  adaptive = FALSE,
  bw = NULL,
  bw_response = NULL,
  bw_approach = "CV",
  include_self = FALSE
)
```

Arguments

data	A data.table/data frame of the SOURCE units, one row per unit, containing unit, the coordinate columns in coords, and all value_cols.
unit	Character; the unit-id column (present in both source and target).
value_cols	Character vector of numeric columns to lag. Each returns a column named <value_col>_LM.
coords	Length-2 character vector naming longitude/latitude columns. Default c("longitude", "latitude").
predict_data	Optional TARGET units (one row per unit) with unit and coords. Defaults to data (targets = sources).
distance_metric	One of gw_distance_metric_names(). Default "Euclidean".
kernel	GW kernel. Default "bisquare".
adaptive	Logical; adaptive (kNN) bandwidth if TRUE. Default FALSE.
bw	Optional pre-computed bandwidth. If NULL, selected once via GWmodel::bw.gwr(bw_response ~ 1) on the source set.
bw_response	Column used for bandwidth selection when bw is NULL. Default: the first value_cols.
bw_approach	Bandwidth criterion. Default "CV".
include_self	Logical; keep the focal unit in its own neighbourhood. Default FALSE (strict neighbour mean).

Value

A data.table with one row per target unit: the unit id and one <value_col>_LM column per value column. The selected bandwidth is attached as attr(., "bandwidth").

See Also

Other Geographically weighted summaries: estimate_gwlag_by_polygon()

---

estimate_gwlag_by_polygon

Geographically weighted spatial lag on polygons

Description

As estimate_gwlag_by_point(), but the spatial units are polygons; centroid coordinates are derived from polygons and joined to data (and, if given, predict_data) by the unit id.

Usage

```
estimate_gwlag_by_polygon(
  data,
  unit,
  polygons,
  value_cols,
  poly_id = unit,
  predict_ids = NULL,
  distance_metric = "Euclidean",
  kernel = "bisquare",
  adaptive = FALSE,
  bw = NULL,
  bw_response = NULL,
  bw_approach = "CV",
  include_self = FALSE
)
```

Arguments

`data`	Source units (one row per unit) with `unit` and all `value_cols`.
`unit`	Character; the unit-id column.
`polygons`	A `terra` `SpatVector` or `sf` polygon layer whose id field (`poly_id`) matches `unit`. Provides target ids and centroids; the value columns are joined from `data` (and `predict_ids` restricts the targets).
`value_cols`	Character vector of numeric columns to lag.
`poly_id`	Name of the id field in `polygons`. Default: value of `unit`.
`predict_ids`	Optional character vector of target unit ids (subset of the polygon ids). Default: all polygon units.
`distance_metric, kernel, adaptive, bw, bw_response, bw_approach, include_self`	See `estimate_gwlag_by_point()`.

Value

A `data.table` with one row per target unit (see `estimate_gwlag_by_point()`).

See Also

Other Geographically weighted summaries: `estimate_gwlag_by_point()`

`estimate_gwr_by_point` *Geographically weighted local mean/variance regression at points*

Description

For each point-referenced unit, fits a geographically weighted regression of `response` on `covariate` (the local slope on the response mean) and, unless disabled, a second GWR of the log-squared residuals (the local slope on the response variance). Local slope standard errors, t-values, and normal- approximation p-values are computed directly (GWmodel returns no SE at supplied regression points). Distances and the fixed bandwidth use GWmodel.

Usage

```
estimate_gwr_by_point(
  data,
  unit,
  response,
  covariate = "trend",
  coords = c("longitude", "latitude"),
  kernel = "bisquare",
  adaptive = FALSE,
  distance_metric = "Great Circle",
  bw = NULL,
  bw_approach = "AICc",
  bw_sample = 1500L,
  chunk_size = 250L,
  eps = 0.00000001,
  variance = TRUE,
  verbose = FALSE
)
```

Arguments

data	A data.table/data frame with one row per observation, containing the unit id (unit), response, covariate, and the two coordinate columns named in coords. Multiple observations per unit (e.g. a time series) are expected for a meaningful slope.
unit	Character string naming the spatial-unit id column.
response	Character string naming the response column.
covariate	Character string naming the covariate (e.g. "trend"). Default "trend".
coords	Length-2 character vector naming the longitude/latitude columns. Default c("longitude", "latitude").
kernel	GW kernel for GWmodel::bw.gwr() and the local weights. One of "gaussian", "exponential", "bisquare", "boxcar", "tricube". Default "bisquare".
adaptive	Logical; adaptive (kNN) bandwidth if TRUE, fixed distance if FALSE. Default FALSE.
distance_metric	One of gw_distance_metric_names(). Default "Great Circle".
bw	Optional pre-computed bandwidth. If NULL it is selected once via GWmodel::bw.gwr() on a random subsample.
bw_approach	Bandwidth criterion for bw.gwr(). Default "AICc".
bw_sample	Observations sampled for bandwidth selection. Default 1500.
chunk_size	Regression points per distance-matrix chunk. Default 250.
eps	Added to squared residuals before the log. Default 1e-8.
variance	Logical; also estimate the local variance slope. Default TRUE.
verbose	Logical; print chunk progress. Default FALSE.

Value

A data.table with one row per unit: term, unit_id, unit_level, mean_estimate, mean_standard_error, mean_t_value, mean_p_value, (when variance) the var_* equivalents, and model_estimator ("gwr").

See Also

Other Geographically weighted regression: estimate_gwr_by_polygon(), estimate_gwr_coefficients_by_point(), estimate_gwr_coefficients_by_polygon()

`estimate_gwr_by_polygon` *Geographically weighted local mean/variance regression on polygons*

Description

As `estimate_gwr_by_point()`, but the spatial units are polygons: centroid coordinates are derived from a supplied geometry (`terra SpatVector` or `sf`) and joined to `data` by the unit id, then the GWR is run on the centroids.

Usage

```
estimate_gwr_by_polygon(
  data,
  unit,
  polygons,
  response,
  covariate = "trend",
  poly_id = unit,
  kernel = "bisquare",
  adaptive = FALSE,
  distance_metric = "Great Circle",
  bw = NULL,
  bw_approach = "AICc",
  bw_sample = 1500L,
  chunk_size = 250L,
  eps = 0.00000001,
  variance = TRUE,
  verbose = FALSE
)
```

Arguments

`data`	A `data.table`/data frame with one row per observation, containing the unit id (`unit`), `response`, and `covariate`.
`unit`	Character string naming the spatial-unit id column in `data`.
`polygons`	A `terra SpatVector` or `sf` polygon layer whose id field (`poly_id`) matches `unit`.
`response`	Character string naming the response column.
`covariate`	Character string naming the covariate. Default `"trend"`.
`poly_id`	Name of the id field in `polygons`. Default: value of `unit`.
`kernel, adaptive, distance_metric, bw, bw_approach, bw_sample, chunk_size, eps, variance, verbose`	See `estimate_gwr_by_point()`.

Value

A data.table with one row per unit (see estimate_gwr_by_point()).

See Also

Other Geographically weighted regression: estimate_gwr_by_point(), estimate_gwr_coefficients_by_point(), estimate_gwr_coefficients_by_polygon()

---

estimate_gwr_coefficients_by_point

Geographically weighted local regression coefficients at points

---

Description

For each target unit, fits lm(formula, weights = w_i) where w_i are GWmodel::gw.weight() weights over the source units, and returns the full per-unit coefficient table. Accepts an arbitrary (multi-covariate) formula and returns exactly what stats::lm() reports.

Usage

```
estimate_gwr_coefficients_by_point(
  data,
  unit,
  formula,
  coords = c("longitude", "latitude"),
  predict_data = NULL,
  distance_metric = "Euclidean",
  kernel = "bisquare",
  adaptive = TRUE,
  bw = NULL,
  bw_response = NULL,
  bw_approach = "CV",
  terms = NULL
)
```

Arguments

data	Source units (data.table/data frame) with the model variables and the coordinate columns in coords.
unit	Character; the unit-id column (present in source and target).
formula	A model formula, e.g. y ~ x or y ~ x1 + x2.
coords	Length-2 character vector naming longitude/latitude columns. Default c("longitude", "latitude").
predict_data	Optional TARGET units with unit and coords. Defaults to data (targets = sources).
distance_metric	One of gw_distance_metric_names(). Default "Euclidean".
kernel	GW kernel. Default "bisquare".
adaptive	Logical; adaptive (kNN) bandwidth if TRUE. Default TRUE.

bw	Optional pre-computed bandwidth. If NULL, selected once via GWmodel::bw.gwr(bw_response ~ 1) on the source set.
bw_response	Column for bandwidth selection when bw is NULL. Default: the response of formula.
bw_approach	Bandwidth criterion. Default "CV".
terms	Optional character vector restricting the returned terms (e.g. c("avail00")). Default: all terms including the intercept.

Value

A data.table with one row per (target unit x term): term, unit_id, unit_level, est, se, tv, pv, model_estimator. The selected bandwidth is attached as attr(., "bandwidth").

See Also

Other Geographically weighted regression: estimate_gwr_by_point(), estimate_gwr_by_polygon(), estimate_gwr_coefficients_by_polygon()

---

estimate_gwr_coefficients_by_polygon

Geographically weighted local regression coefficients on polygons

---

Description

As estimate_gwr_coefficients_by_point(), but the spatial units are polygons; centroids are derived from polygons and joined by the unit id.

Usage

```
estimate_gwr_coefficients_by_polygon(
  data,
  unit,
  polygons,
  formula,
  poly_id = unit,
  predict_ids = NULL,
  distance_metric = "Euclidean",
  kernel = "bisquare",
  adaptive = TRUE,
  bw = NULL,
  bw_response = NULL,
  bw_approach = "CV",
  terms = NULL
)
```

Arguments

data	Source units with the model variables.
unit	Character; the unit-id column.

polygons	A terra SpatVector or sf polygon layer whose id field (poly_id) matches unit. Provides target ids and centroids.
formula	A model formula, e.g. y ~ x1 + x2.
poly_id	Name of the id field in polygons. Default: value of unit.
predict_ids	Optional character vector of target unit ids. Default: all polygon units.
distance_metric, kernel, adaptive, bw, bw_response, bw_approach, terms	See estimate_gwr_coefficients_by_point().

Value

A data.table with one row per (target unit x term) (see estimate_gwr_coefficients_by_point()).

See Also

Other Geographically weighted regression: estimate_gwr_by_point(), estimate_gwr_by_polygon(), estimate_gwr_coefficients_by_point()

---

estimate_gwss_by_county

Estimate geographically weighted summary statistics (GWSS) for counties

---

Description

Computes Geographically Weighted Summary Statistics (GWSS) for one or more scalar, county-level variables observed in a subset of counties, then evaluates the statistics at **all** county locations (points-on-surface). This is useful for spatial smoothing and gap-filling (imputation) when some counties are missing observations.

Usage

```
estimate_gwss_by_county(
  data,
  fip_col,
  variable_list,
  distance_metric = "Euclidean",
  kernel = "gaussian",
  target_crs = 5070,
  draw_rate = 0.5,
  approach = "CV",
  adaptive = TRUE,
  state_fips_limits = NULL
)
```

Arguments

data	A data.frame/data.table with at least fip_col and all columns listed in variable_list.
fip_col	Character. Name of the county ID column in data; copied to "county_fips" for joining with the county map.

variable_list Character vector. Names of one or more numeric columns in data to summarize using GWSS.

distance_metric Character. One of gw_distance_metric_names(). Default: "Euclidean".

kernel Character. One of "gaussian", "exponential", "bisquare", "boxcar", "tricube". Default: "gaussian".

target_crs Integer EPSG code used to project county geometries when longlat = FALSE. Default: **5070** (NAD83 / CONUS Albers, meters).

draw_rate Numeric in (0, 1]. Fraction of observed counties used during bandwidth cross-validation. Default: **0.5** (50%).

approach Character. Bandwidth selection approach passed to GWmodel::bw.gwr(). One of "CV", "AIC", "AICc". Default: "CV".

adaptive Logical. Use adaptive (nearest-neighbour count) bandwidth instead of fixed distance. Default: TRUE.

state_fips_limits Optional character or numeric vector of state FIPS codes. When supplied, GWSS is evaluated only for counties whose state_fips is in this set.

Details

The function:

1. pulls U.S. counties from **urbnmapr** and projects to a suitable CRS;
2. copies data[[fip_col]] into "county_fips" and joins to the map;
3. identifies **observed counties** (finite values of the first entry in variable_list), fits GWSS using those points, and selects an adaptive bandwidth by cross-validation on a random subsample sized by draw_rate;
4. optionally restricts the evaluation grid to a subset of states via state_fips_limits;
5. evaluates GWSS at **all counties in the evaluation set** and returns local summaries keyed by county_fips.

Inputs: data must contain a county identifier column referenced by fip_col and at least one numeric column referenced in variable_list. Internally, a column "county_fips" is created for joining with urbnmapr::get_urbn_map("counties"). The data are reduced to a single row per county (by county_fips) before fitting.

Multiple variables: variable_list can contain one or more variable names. Bandwidth selection is performed using the first variable in variable_list, but the resulting bandwidth is then used to compute GWSS for all variables in variable_list via GWmodel::gwss().

Distance metric (distance_metric) defines (p, theta, longlat) for GWmodel::gw.dist() via resolve_distance_metric(). Use gw_distance_metric_names() to list valid options. If longlat = TRUE (e.g., "Great Circle"), counties are transformed to EPSG:4326; otherwise they are projected to target_crs (default 5070, meters).

Bandwidth selection: cross-validation (approach = "CV" by default) is run on a uniform random subsample of size ceiling(draw_rate * n_obs), bounded to [5, n_obs - 1], where n_obs is the number of counties with finite values for the first variable in variable_list. Call set.seed() beforehand for reproducibility. The function returns NULL (with a message) if fewer than 5 counties have finite values.

State filtering: if state_fips_limits is non-NULL, the evaluation grid is restricted to counties whose state_fips is in state_fips_limits. This affects both the set of locations at which GWSS is evaluated and the returned county_fips set.

Value

A `data.table` of GW summary statistics for **all counties in the evaluation set**, with a `county_fips` column for merging back to polygons. Column names follow **GWmodel** conventions for each variable in `variable_list`, e.g. for a variable `x`, columns such as `x_LM`, `x_LSD`, `x_LCV`, `x_LSKe`, `x_LSSke`, etc. are returned (depending on `GWmodel::gwss()` defaults). Attributes attached: `"bandwidth"`, `"distance_params"`, `"kernel"`, `"approach"`, `"adaptive"`. Returns `NULL` when there are fewer than 5 observed counties.

`estimate_gwss_by_point` *Estimate geographically weighted summary statistics (GWSS) for points*

Description

Computes geographically weighted summary statistics (via **GWmodel::gwss**) at specified prediction locations using a **single global bandwidth** determined from a subsample of observed points. All observed points are used for every prediction; `group_variable` only partitions the prediction set for organizational or computational reasons.

Usage

```
estimate_gwss_by_point(
  locat_observed,
  locat_predict,
  longitude_col,
  latitude_col,
  variable,
  distance_metric = "Euclidean",
  kernel = "gaussian",
  target_crs = 5070,
  draw_rate = 0.5,
  approach = "CV",
  adaptive = TRUE,
  feasible_radius = 100,
  group_variable = NULL,
  identifiers = NULL
)
```

Arguments

`locat_observed`	`data.frame`/`data.table` with columns named by `longitude_col`, `latitude_col`, and a numeric column named by `variable`. Must be in decimal degrees (EPSG:4326).
`locat_predict`	`data.frame`/`data.table` with columns named by `longitude_col` and `latitude_col`. Must be in decimal degrees (EPSG:4326). Rows with no nearby observations (within `feasible_radius`) are dropped.
`longitude_col, latitude_col`	`character` column names for coordinates.
`variable`	`character` name of numeric column in `locat_observed` to summarize.

distance_metric
: character distance metric passed to GWmodel::gw.dist() via resolve_distance_metric(). Default "Euclidean".

kernel
: character kernel for GWR/GWSS weights. One of "gaussian", "exponential", "bisquare", "boxcar", "tricube". Default "gaussian".

target_crs
: integer EPSG code used for projection during radius pre-screening (in meters). Default 5070 (NAD83 / CONUS Albers).

draw_rate
: numeric fraction (0,1] of observed points used for bandwidth selection. Default 0.5.

approach
: character one of "CV", "AIC", "AICc", passed to GWmodel::bw.gwr(). Default "CV".

adaptive
: logical whether to use adaptive (kNN) bandwidth. Default TRUE.

feasible_radius
: numeric radius in **miles** for pre-screening prediction locations. Default 100.

group_variable
: character or NULL. Optional column in locat_predict used to partition prediction points; observed points are always global.

identifiers
: character vector of column names from locat_predict to carry through to the output.

Details

- Coordinates are assumed to be in **WGS84 (EPSG:4326)**.
- Prediction points without any observed points within feasible_radius miles are excluded.
- Bandwidth is estimated **once** via GWmodel::bw.gwr() on a random subsample (size = draw_rate * n_obs, bounded to 5, n_obs - 1).
- group_variable, if provided, splits the prediction set for parallel or modular computation, but the same global bandwidth and all observed points are used for each group.

Value

A data.table containing:

- GWSS summary statistics (e.g., local mean, SD, etc.)
- longitude, latitude - coordinates of prediction points
- block_variable - group label or 1L
- bandwidth - estimated global bandwidth
- p, theta, longlat, approach, adaptive, variable - metadata
- any columns from identifiers

Returns an empty data.table if no prediction locations pass the radius screen.

Implementation notes

- Coordinates must be valid WGS84 (longitude = x, latitude = y).
- Radius pre-screen is computed using projected distances (in meters).
- Global bandwidth ensures comparability across groups.
- Random seed (set.seed(1L)) ensures reproducibility of CV subsampling.

References

Fotheringham, A. S., Brunsdon, C., & Charlton, M. (2002). *Geographically Weighted Regression: The Analysis of Spatially Varying Relationships.* John Wiley & Sons.

`estimate_gwss_by_polygon`

Estimate geographically weighted summary statistics for polygons

---

Description

This function estimates Geographically Weighted Summary Statistics (GWSS) for one or more numeric variables observed for a subset of polygons, then evaluates the local summary statistics at all polygon locations in a supplied spatial polygon layer. It is useful for spatial smoothing and gap-filling when some polygons do not have observed values.

Usage

```
estimate_gwss_by_polygon(
  data,
  shape_file,
  fip_col,
  variable_list,
  distance_metric = "Euclidean",
  kernel = "gaussian",
  target_crs = 5070,
  draw_rate = 0.5,
  approach = "CV",
  adaptive = TRUE
)
```

Arguments

`data`	A `data.frame` or `data.table` containing the polygon identifier column specified by `fip_col` and all numeric columns listed in `variable_list`.
`shape_file`	An `sf` polygon object containing the polygon geometries. This object must contain the polygon identifier column specified by `fip_col`.
`fip_col`	Character. Name of the polygon identifier column shared by `data` and `shape_file`. This column is copied internally to `polygon_fips` and used for joining the observed data to the polygon layer.
`variable_list`	Character vector. Names of one or more numeric columns in `data` to summarize using GWSS. The first variable in this vector is also used to define the observed polygons and to select the bandwidth.
`distance_metric`	Character. Name of the distance metric to use. Must be one of the values supported by `gw_distance_metric_names()`. The selected metric is resolved by `resolve_distance_metric()` into the `p`, `theta`, and `longlat` arguments used by `GWmodel`.
`kernel`	Character. Kernel function passed to `GWmodel::bw.gwr()` and `GWmodel::gwss()`. Must be one of `"gaussian"`, `"exponential"`, `"bisquare"`, `"boxcar"`, or `"tricube"`. Default is `"gaussian"`.

target_crs	Integer EPSG code used to project polygon geometries when the selected distance metric does not use longitude/latitude distances. Default is 5070, NAD83 / CONUS Albers.
draw_rate	Numeric value in (0, 1]. Fraction of observed polygons used for bandwidth cross-validation. The actual subsample size is bounded between 5 and n_obs - 1, where n_obs is the number of observed polygons. Default is 0.5.
approach	Character. Bandwidth selection criterion passed to GWmodel::bw.gwr(). Must be one of "CV", "AIC", or "AICc". Default is "CV".
adaptive	Logical. If TRUE, use an adaptive nearest-neighbor bandwidth. If FALSE, use a fixed-distance bandwidth. Default is TRUE.

Details

The function first prepares the input data by copying the polygon identifier specified by fip_col into a common internal column named polygon_fips. It then keeps one finite observation per polygon, joins those observations to the supplied shape_file, and uses the observed polygons to fit GWSS. The estimated geographically weighted summaries are then evaluated at all polygon locations using points-on-surface.

The function is designed for polygon-level spatial smoothing. Observed values are taken from data, while the full set of possible evaluation locations comes from shape_file. This means the returned object can include polygons that were not present in data, with their local summaries estimated from nearby observed polygons.

Internally, the function:

1. validates the input data, variables, kernel, and bandwidth-selection approach;
2. resolves the selected distance metric using resolve_distance_metric();
3. creates a generic polygon identifier, polygon_fips, from fip_col;
4. keeps one finite observation per polygon based on the first variable in variable_list;
5. transforms the polygon layer to EPSG:4326 when longlat = TRUE, or to target_crs otherwise;
6. joins the observed data to the polygon geometries while retaining all polygons in shape_file;
7. selects a bandwidth using GWmodel::bw.gwr() on a random subsample of observed polygons;
8. computes GWSS using observed polygons as the data locations and all polygons in shape_file as the summary locations.

Bandwidth selection uses only the first variable in variable_list, but the selected bandwidth is then used to compute GWSS for all variables in variable_list. Use set.seed() before calling this function if reproducible subsampling is needed.

The function returns NULL with a message when fewer than five polygons have finite observed values for the first variable in variable_list.

Value

A data.table containing geographically weighted summary statistics evaluated at all polygons in shape_file. The returned table includes the polygon identifier column and GWSS output columns following GWmodel::gwss() naming conventions. For a variable named x, returned columns may include local statistics such as x_LM, x_LSD, x_LCV, x_LSKe, and related GWSS summaries, depending on GWmodel::gwss() defaults.

The returned object also has the following attributes:

bandwidth The selected GW bandwidth.

distance_params A list containing `p`, `theta`, and `longlat`.

kernel The kernel used for bandwidth selection and GWSS.

approach The bandwidth selection approach.

adaptive Whether an adaptive bandwidth was used.

`gw_distance_metric_names` *List valid GW distance metric names*

Description

List valid GW distance metric names

Usage

```
gw_distance_metric_names()
```

Value

Character vector of valid preset names.

`gw_distance_metric_presets` *GW distance metric presets for GWmodel*

Description

Provides a curated set of distance metric presets (Minkowski family and great-circle) for **GWmodel**. Each preset specifies `(p, theta, longlat)` for `GWmodel::gw.dist()`.

Usage

```
gw_distance_metric_presets()
```

Value

A named list of presets, each entry a `list(p, theta, longlat)`.

gw_optimal_class_by_point

Optimal class for point (lattice) units

Description

Collapses a stacked table of per-unit classifications (one row per unit per `distance_metric` x `kernel` setting, within a history window) to a single optimal class per unit: the mode across settings (`modal_class`) plus a first-order Queen contiguity vote for optional spatial de-speckling (`queen_class`), with ties resolved by widening the Queen order. Coordinates are read directly from the table, matching `estimate_gwss_by_point()`.

Usage

```
gw_optimal_class_by_point(
  class_dt,
  unit_col,
  class_col = "trend_class_05",
  coords = c("longitude", "latitude"),
  include_self = TRUE,
  max_order = 10L,
  class_levels = NULL,
  verbose = FALSE
)
```

Arguments

`class_dt`	A `data.table`/data frame stacked over settings, containing the unit id (`unit_col`), the categorical class (`class_col`), and the two coordinate columns named in `coords`.
`unit_col`	Name of the spatial-unit identifier column (e.g. `"grid_id"`).
`class_col`	Name of the categorical class column. Default `"trend_class_05"`.
`coords`	Length-2 character vector naming the longitude/latitude columns. Default `c("longitude", "latitude")`.
`include_self`	Logical; count the unit itself in the Queen vote. Default `TRUE`.
`max_order`	Maximum Queen order to expand to when breaking ties. Default `10`.
`class_levels`	Optional character vector of all class labels (fixes the count columns / vote ordering). Default: sorted unique observed classes.
`verbose`	Logical; print progress. Default `FALSE`.

Value

A `data.table` with one row per unit: the id column, `longitude`, `latitude`, `n_settings`, `modal_class`, `modal_agreement`, `queen_class`, `queen_order` (order at which the vote resolved), and `queen_agreement`.

See Also

Other Optimal class selection: `gw_optimal_class_by_polygon()`

gw_optimal_class_by_polygon *Optimal class for polygon units*

Description

As gw_optimal_class_by_point(), but the spatial units are polygons: the Queen neighbours are shared-boundary contiguities of a supplied geometry (matching estimate_gwss_by_polygon(), keyed by e.g. polygon_fips), and centroids for the output come from the same geometry. Accepts a terra SpatVector or an sf object.

Usage

```
gw_optimal_class_by_polygon(
  class_dt,
  unit_col,
  polygons,
  class_col = "trend_class_05",
  poly_id = unit_col,
  include_self = TRUE,
  max_order = 10L,
  class_levels = NULL,
  verbose = FALSE
)
```

Arguments

class_dt	A data.table/data frame stacked over settings, containing the unit id (unit_col) and the categorical class (class_col).
unit_col	Name of the spatial-unit identifier column (e.g. "polygon_fips").
polygons	A terra SpatVector or sf polygon layer whose id field (poly_id) matches unit_col.
class_col	Name of the categorical class column. Default "trend_class_05".
poly_id	Name of the id field in polygons. Default: value of unit_col.
include_self, max_order, class_levels, verbose	See gw_optimal_class_by_point().

Value

A data.table with one row per unit (see gw_optimal_class_by_point()).

See Also

Other Optimal class selection: gw_optimal_class_by_point()

gw_optimal_scalar_by_point *Consensus scalar for point (lattice) units*

Description

Collapses a stacked table of per-unit continuous estimates (one row per unit per `distance_metric` x `kernel` setting, within a history window) to a single per-unit consensus value via a user-supplied reducer (`agg_fun`, default the median), together with the spec-uncertainty spread and a sign-agreement share. Optionally reconciles each unit's consensus with its first-order Queen neighbourhood, widening the Queen order until enough neighbours are gathered. Coordinates are read directly from the table, matching `estimate_gwss_by_point()`.

Usage

```
gw_optimal_scalar_by_point(
  value_dt,
  unit_col,
  value_col = "estimate",
  coords = c("longitude", "latitude"),
  agg_fun = stats::median,
  probs = c(0.05, 0.95),
  queen_smooth = FALSE,
  include_self = TRUE,
  min_count = 1L,
  max_order = 10L,
  verbose = FALSE
)
```

Arguments

`value_dt`	A `data.table`/data frame stacked over settings, containing the unit id (`unit_col`), the continuous estimate (`value_col`), and the two coordinate columns named in `coords`.
`unit_col`	Name of the spatial-unit identifier column (e.g. `"grid_id"`).
`value_col`	Name of the continuous value column. Default `"estimate"`.
`coords`	Length-2 character vector naming the longitude/latitude columns. Default `c("longitude", "latitude")`.
`agg_fun`	Function reducing a numeric vector (the values across settings for one unit) to a single scalar. Default `stats::median`. Any function of the form `function(x) ...` works, e.g. `mean`, `function(x) mean(x, trim = 0.1)`.
`probs`	Length-2 numeric; the lower/upper quantiles reported across settings as `consensus_lo`/`consensus_h` Default `c(0.05, 0.95)`.
`queen_smooth`	Logical; if `TRUE`, also compute the Queen-neighbourhood smoothed consensus. Default `FALSE`.
`include_self`	Logical; include the unit itself in the Queen neighbourhood. Default `TRUE`.
`min_count`	Minimum number of units to gather before reducing the Queen neighbourhood; the order widens until reached. Default `1`.
`max_order`	Maximum Queen order to expand to. Default `10`.
`verbose`	Logical; print progress. Default `FALSE`.

Value

A data.table with one row per unit: the id column, longitude, latitude, n_settings, consensus, consensus_sd, consensus_mad, consensus_lo, consensus_hi, sign_agreement, and - when queen_smooth = TRUE - queen_value, queen_order, queen_agreement.

See Also

Other Consensus scalar: gw_optimal_scalar_by_polygon()

---

gw_optimal_scalar_by_polygon *Consensus scalar for polygon units*

Description

As gw_optimal_scalar_by_point(), but the spatial units are polygons: the Queen neighbours are shared-boundary contiguities of a supplied geometry (matching estimate_gwss_by_polygon(), keyed by e.g. polygon_fips), and centroids for the output come from the same geometry. Accepts a terra SpatVector or an sf object.

Usage

```
gw_optimal_scalar_by_polygon(
  value_dt,
  unit_col,
  polygons,
  value_col = "estimate",
  poly_id = unit_col,
  agg_fun = stats::median,
  probs = c(0.05, 0.95),
  queen_smooth = FALSE,
  include_self = TRUE,
  min_count = 1L,
  max_order = 10L,
  verbose = FALSE
)
```

Arguments

value_dt	A data.table/data frame stacked over settings, containing the unit id (unit_col) and the continuous estimate (value_col).
unit_col	Name of the spatial-unit identifier column (e.g. "polygon_fips").
polygons	A terra SpatVector or sf polygon layer whose id field (poly_id) matches unit_col.
value_col	Name of the continuous value column. Default "estimate".
poly_id	Name of the id field in polygons. Default: value of unit_col.
agg_fun, probs, queen_smooth, include_self, min_count, max_order, verbose	See gw_optimal_scalar_by_point().

Value

A `data.table` with one row per unit (see `gw_optimal_scalar_by_point()`).

See Also

Other Consensus scalar: `gw_optimal_scalar_by_point()`

`PACKAGE_GLOBALVARIABLES` *global_names*

Description

A combined dataset for global_names

Usage

```
PACKAGE_GLOBALVARIABLES
```

Format

A list of global names.

Source

Internal innovation

`resolve_distance_metric` *Resolve a GW distance metric preset*

Description

Resolve a GW distance metric preset

Usage

```
resolve_distance_metric(name, stop_on_error = TRUE)
```

Arguments

`name`	Character scalar. One of `gw_distance_metric_names()`.
`stop_on_error`	Logical. If `TRUE`, throw for unknown names; else `NULL`.

Value

`list(p, theta, longlat)` or `NULL`.

Index